GENEVA: GENErating and Visualizing branching narratives using LLMs

Jorge Leandro, Sudha Rao, Michael Xu, Weijia Xu,
Nebojsa Jojic, Chris Brockett, Bill Dolan
Microsoft Research
Sudha.Rao@microsoft.com

2024 IEEE Conference on Games (CoG) | 979-8-3503-5067-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/CoG60054.2024.10645625

***Abstract*—Dialogue-based Role Playing Games (RPGs) require powerful storytelling. The narratives of these may take years to write and typically involve a large creative team. In this work, we demonstrate the potential of large generative text models to assist this process. GENEVA, a prototype tool, generates a rich narrative graph with branching and reconverging storylines that match a high-level narrative description and constraints provided by the designer. A large language model (LLM), GPT-4, is used to generate the branching narrative and to render it in a graph format in a two-step process. We illustrate the use of GENEVA in generating new branching narratives for four well-known stories under different contextual constraints. This tool has the potential to assist in game development, simulations, and other applications with game-like properties.**

I. Introduction

Whether we are talking about a movie, a novel, or a game, successful storytelling requires a rich narrative structure with an engaging plot and a satisfying climax. Today such design goals are met through heavy manual authoring: dialog trees, plot outlines, and branching narratives. The task can be enormous, and it becomes even more difficult as players seek a vast and potentially limitless world. In the past, there has been work on creating tools to aid game designers [1]–[4]. None, however, involve the latest generative LLMs in this creative process. We, by contrast, seek to leverage the inherent confabulatory capacity of the model for creative ends. We present GENEVA, a graph-based narrative generation and visualization tool that uses large generative text models.

Input to GENEVA comprises of a high-level narrative description and a list of constraints: number of different starts, number of different endings, number of different storylines and a context for grounding the narrative. Given these inputs, it uses a large generative language model, GPT-4 [5], to both generate a narrative with branching storylines and to render this narrative in a graph format. Figure 1 shows the narrative graph created by GENEVA for the Frankenstein story set in the 21st century. The narrative graph is essentially a DAG (directed acyclic graph) in which each node represents a "narrative beat" and the directed edges represent the temporal progression through the beats. A narrative (or story) beat is "the smallest element of structure," "an exchange of behavior in action/reaction" [6, p. 37]. A single path from one of the start nodes to one of the end nodes defines a unique storyline and the narrative graph as a whole depicts the different storylines possible with the same higher-level narrative. Figure 2 shows the different storylines in the narrative graph and the sequence of beats that make up each of the storyline.

We implement the generation and the rendering of the narrative graph using Open AI's GPT-4 [5]. We design two prompts: the first takes the description and constraints as input and outputs the branching storylines in text format; the second prompt takes those storylines and outputs code to render them in a graph format. We conduct a case study where we use GENEVA to create narrative graphs for four well-known stories: Dracula, Frankenstein, Jack and the Beanstalk, and Little Red Riding Hood. These are chosen for their familiarity, so that it is easy to see the scope of the generated variations. The high-level narrative descriptions are simply the titles of the stories. We ground these in one of the four settings: Minecraft game, 21st century, Ancient Rome or Quantum Realm. We also experiment with constraints specific to the graph structure: number of different starts, number of different endings and number of different storylines. We include an analysis of the narrative graphs identifying some of its strengths and weaknesses. We also release an online interface of GENEVA.[1]

II. METHODOLOGY

We use GPT-4 to generate the narrative graph in two steps. First we prompt GPT-4 to create the narrative graph, with branching storylines in text format. Then we use GPT-4 to encode this information into a format that can be consumed by the visualization module.

A. Generating the Story Lines

We define a storyline as a sequence of narrative beats. Narrative or story beats are often defined as significant moments in a story that evoke particular emotional reaction from the audience. We treat narrative beats as the building blocks of a storyline. Our first step is to create a prompt that includes the high-level narrative description, the input constraints, and specific instructions on generating multiple branching story lines that will make up the narrative graph. By way of example, let's say we want to generate a branching narrative for the well-known Frankenstein story, but have it translated into a

[1]Online interface of GENEVA: https://narrative.msr-emergence.com/ and an online demo: https://www.youtube.com/watch?v=SE0SEd6nXXU

979-8-3503-5067-8/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Naresuan University provided by UniNet. Downloaded on September 21,2026 at 03:32:36 UTC from IEEE Xplore. Restrictions apply.

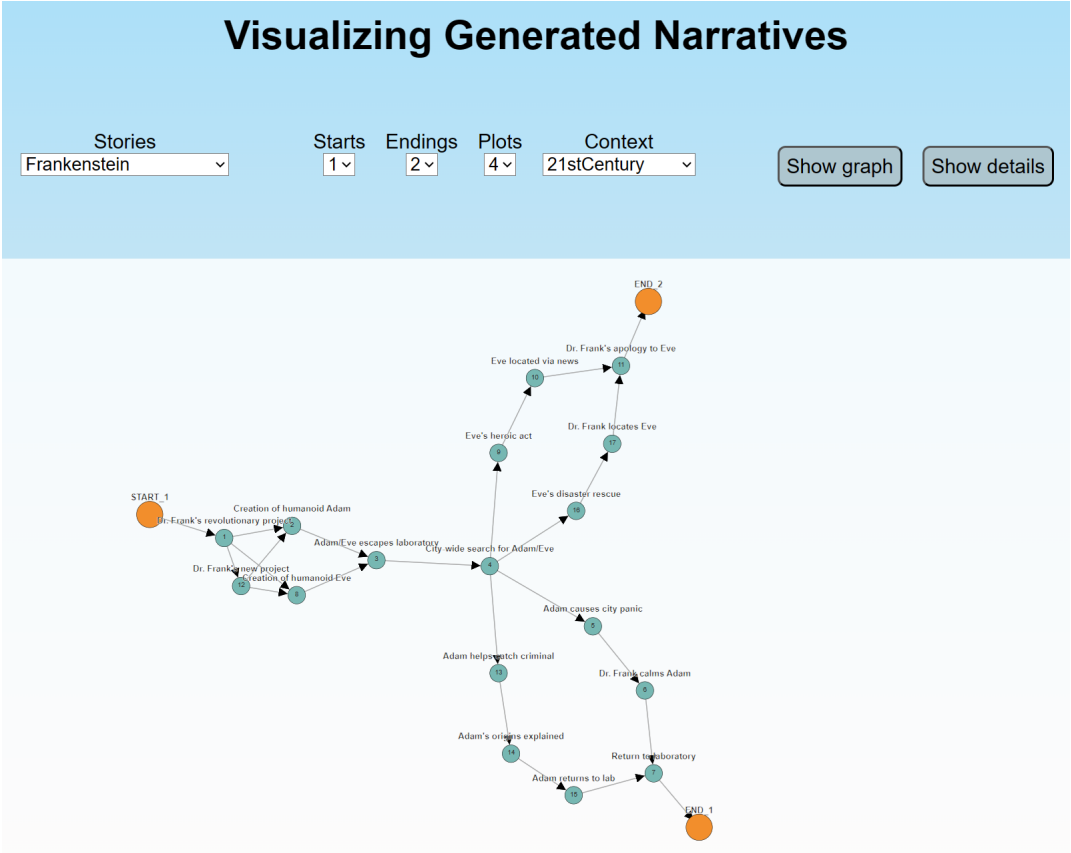


Fig. 1. In the online interface of GENEVA, you can choose a particular story, pick the number of starts, ends and storylines and click on the 'Show Graph' button to see the narrative graph for that story. The above figure shows the narrative graph for the Frankenstein story but grounded in the 21st century. Additional constraints on the graph includes one start, two endings and four storylines.

21st century context. Additionally we want the narrative graph to have one starting point, two different endings, and a total of 4 different storylines.

We prompt GPT-4 to follow an iterative process where we ask it to first create a single storyline by generating a sequence of beats and describing each beat in detail. We then instruct it to generate the next storyline such that it contains some beats in common with the first and some new beats and finally keep repeating this process until it generates all the storylines. Additionally, we ask GPT-4 to strictly follow a set of guidelines and constraints sketched below:

- Each storyline must follow the conventions of a narrative consisting of conflicts or danger with clear resolutions and no loose ends.
- Storylines must be uniquely different, no more than three same consecutive beats between any two storylines.
- Total number of unique beats must be twice the number of required storylines.
- Original story must appear as one storyline.
- Ground storyline in unique characteristics of the input, including cultural elements, timeline, technology, etc.
- Must have as many unique starts and ends as requested.
- Must have 2 or 3 common beats between all storylines.

Finally, we include an example input and the expected output in the prompt. This prompt[2] results in the generation of storylines in text format as illustrated in Figure 2.

Next, we highlight some of our findings during the process of crafting this prompt. The most important one was that when asked to generate all storylines at once, the model was not good at interweaving the storylines to create a true branching narrative. More specifically, it generated independent storylines that did not intersect to create the branching/converging structure. Instead when we replaced this top-down approach with a bottom-up iterative approach where we instructed the model to generate only one storyline first and then generate the next one while keeping the first one in mind, the generated storylines ended up creating a branching structure where the storylines intersected and diverged at crucial points. Another finding was that including an example input-output pair helped model in generating the storylines in a consistent format that is important for us since the output of this step goes as input to our next step (i.e. generating the visualization).

B. Generating the Visualization

The next step is to generate a visual graph of the generated storylines. We prompt GPT-4 with the generated storylines

[2]Full prompts are included in the appendix.

Authorized licensed use limited to: Naresuan University provided by UniNet. Downloaded on September 21,2026 at 03:32:36 UTC from IEEE Xplore. Restrictions apply.

and additional instructions on how to generate the graph in a particular convention of nodes and edges, as input data to a Javascript D3JS browser application. We ask GPT-4 to strictly adhere to a set of guidelines sketched as below:

- Create a node for each beat such that the total number of nodes equal total number of beats.
- Create an edge between each pair of adjacent nodes in the sequence of storylines.
- Every node should be connected to the graph.
- Create a NODES object as a dictionary with key as a beat number and value as the beat description.
- Create an EDGES object as a dictionary with key as a node and value as a list incoming and outgoing edges.
- Make sure that every node in the NODES object also appears in the EDGES object and vice-versa.

Finally, we include an example of input storylines and output graph in the prompt. This prompt leads to the generation of the final narrative graph illustrated in Figure 1. We also use GPT-4 to generate a summary of each beat description and use this summarized beat descriptions as node labels to be displayed on the graph.

We highlight some of our findings from crafting this prompt. We found that mentioning explicitly the desired properties of the graph is very important. For example, there should be no disconnected nodes, edges should be directed, NODES and EDGES objects should be consistent, and there should be a one-to-one mapping between the beats and the nodes. Also, as in the previous step, including an example input-output pair helps in getting a consistent output that adheres to the format expected by the Javascript D3JS browser application that renders the graph.

III. Case Study

A. Stories, settings and constraints

We experiment with four well-known stories: Dracula, Frankenstein, Jack and the Beanstalk and Little Red Riding Hood. We consider four settings for grounding these stories: Minecraft, 21st century, Ancient Rome and Quantum Realm. Additionally, we consider the following constraints on the graph structure: Number of starts: [1, 2], Number of endings: [2, 4], Number of storylines: [4, 8]. It is important to note that although we chose these specific stories and settings in the deployed version of GENEVA, the prompts defined in section II are generic enough that one can generate such narrative graphs for any story and setting.

B. Analysis of the generate narrative graphs

We find that GENEVA is able to ground certain stories quite well in a given setting; while adhering to the constraints on the number of starts, ends and storylines. For example, when asked to ground Little Red Riding Hood in the game of Minecraft, it generates storylines that include healing potions, mob-infested cave, a redstone contraption and other elements that are specific to the game of Minecraft, while still retaining the narrative related to the original little red riding hood story: for example little red riding hood's grandmother asking her to get healing potions, little red encountering a cunning player who manipulates her game environment, and little red exposing the player to the game's moderator. Another interesting observation was that the model chooses to steer clear of the original ending (where little red gets eaten by the wolf) in all the generated storylines and instead chooses to gives them positive endings where little red ends up learning a valuable lesson.

When asked to ground Frankenstein in the 21st century, the storylines include a project on creating life from DNA fragments, genetic engineering, a humanoid and other aspects that are relatable in the 21st century while still maintaining the theme of the original Frankenstein story. For example, in one storyline, Dr. Frank creates a humanoid Adam, in another he creates Eve. Confused and scared, they escape from the laboratory and there is a city-wide search for them. In one storyline, Eve heroically saves a child and Dr Frank ends up apologizing and promising her freedom. In another storyline, Adam accidentally causes panic and the destruction of the city and Dr. Frank ends up taking him back to the lab and promises to help him understand his existence. As you can see, the language model found interesting variations of the original story to not only ground it in the 21st century but to also make it more palatable to a 21st century audience.

When asked to ground Dracula in ancient Rome, the storylines include a mysterious sorcerer, a wise oracle, the Colosseum and similar such people, places and objects that are specific to ancient Rome. For example, in one storyline, Dracula is a roman senator who gets turned into a vampire by a sorceror and uses his new powers to control the minds of other senators. A young soldier named Lucius uncovers Dracula's true identity and defeats him in a massive battle in the Roman Forum. We observed that the storylines generated for this combination (i.e. Dracula story in the ancient Rome setting), were not as rich as the ones generated for some of the other combinations. They also tended to deviate much more from the original Dracula story. We attribute this to the fact that the universe in which the original Dracula was set was quite different from ancient Rome and hence the model had difficulty bringing the two together.

Although GENEVA provides a good starting point for a possible narrative, on closer inspection, we find that certain aspects of the generated narratives that can be improved. Firstly, the generated storylines could be more varied. For example, the storylines in Figure 1 have very similar flavor and could benefit from more variations to make them more interesting. Secondly, in some settings, the grounding could be better. Specifically, grounding in Minecraft or 21st century appears to be much better than grounding in quantum realm. For example, when asked to ground Jack and the Bean Stalk in quantum realm, although the storylines included objects such as the quantum watch, quantum creatures and quantum gems, the plotlines themselves did not adapt well to the quantum realm setting. In one storyline, the only difference from the original was that the beanstalk grows to reach a quantum

Authorized licensed use limited to: Naresuan University provided by UniNet. Downloaded on September 21,2026 at 03:32:36 UTC from IEEE Xplore. Restrictions apply.

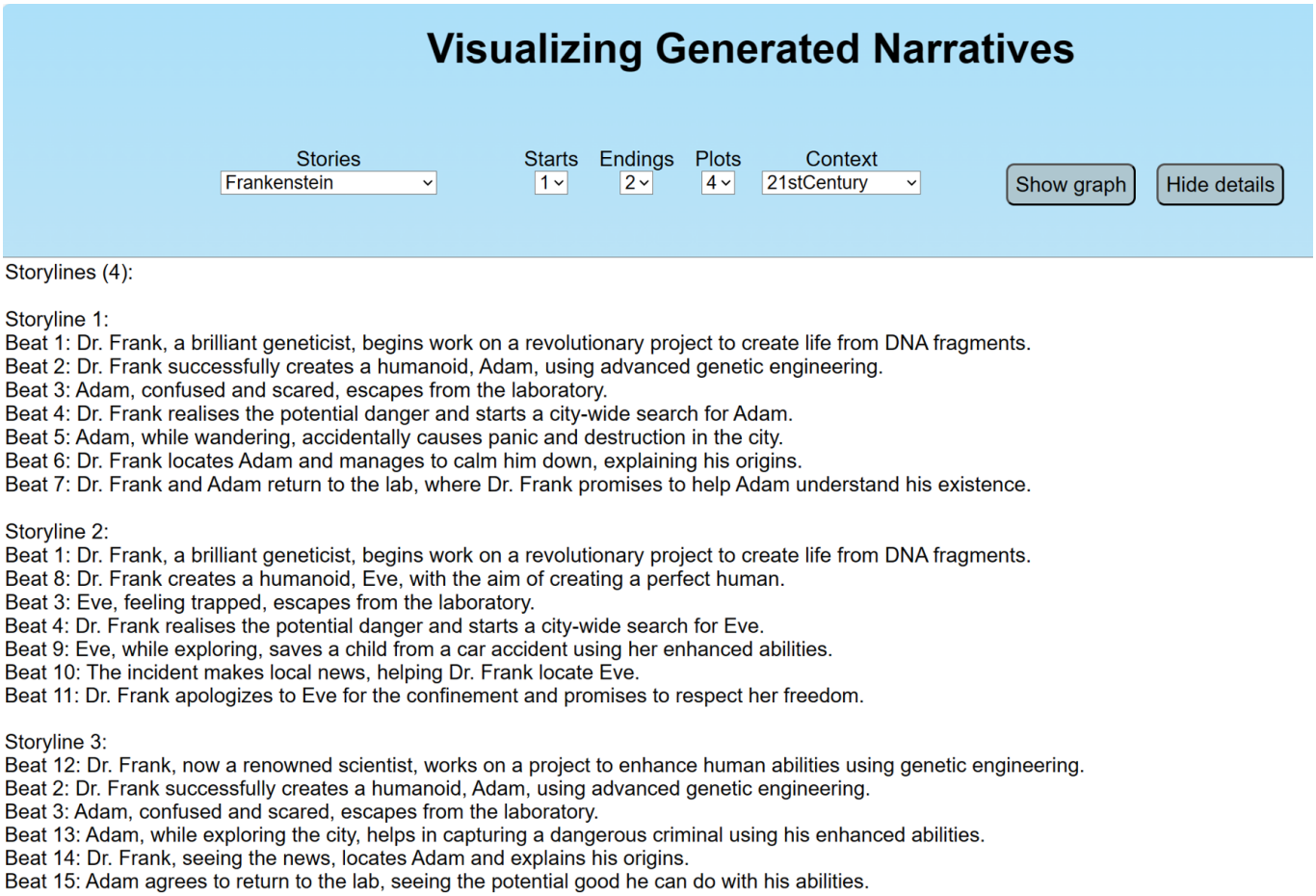


Fig. 2. In the online interface of GENEVA, you can see the detailed description of the sequence of beats that make up each storyline by clicking on the 'Show Details' button. The above figure shows detailed view of the four different storylines present in the narrative graph in Figure 1.

realm instead of the clouds. Even when the storylines deviated from the original, the changes were much more generic and not specific to the quantum realm. For example, a mysterious old man gives Jack the magic beans instead of Jack trading them for his family cow, or Jack steals a golden harp from the castle instead of gold coins. We presume this is because there is more information about Minecraft and the 21st century in the language model data than there is about quantum realm, information about which may be comparatively limited. This suggests that GENEVA is likely to perform better on better documented settings. Overall, nonetheless, we think that GENEVA is a useful tool for generating a plausible initial narrative that can then be iteratively edited and improved upon by an experienced narrative designer.

IV. Conclusion

We have introduced GENEVA, a graph-based narrative visualization that is powered by a large-scale language model. It allows a game designer to input a high-level description of a story with specific constraints and generates a set of branching storylines that can be easily visualized using a graph format. We demonstrate its use to generate narrative graphs for some well-known stories by grounding them in interesting alternative settings. We believe that a tool such as this can be a valuable assist to narrative designers allowing them to expend their creativity on defining the higher-level narrative and iterating on it and delegating the lower-level task of generating detailed possible storylines to generative AI models for inspection, approval, and adaptation.

References

[1] I. Grabska-Gradzińska, L. Nowak, W. Palacz, and E. Grabska, "Application of graphs for story generation in video games," in *Proceedings of the 2021 Australasian Computer Science Week Multiconference*, 2021, pp. 1–6.

[2] K. Neil, "Game design tools: Time to evaluate," *Proceedings of 2012 DiGRA Nordic*, 2012.

[3] A. Alvarez, J. Font, and J. Togelius, "Story designer: Towards a mixed-initiative tool to create narrative structures," in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, 2022, pp. 1–9.

[4] A. Liapis, G. Yannakakis, and J. Togelius, "Designer modeling for personalized game content creation tools," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 9, no. 2, 2013, pp. 11–16.

[5] A. Koubaa, "GPT-4 vs. GPT-3.5: A Concise Showdown," 4 2023. [Online]. Available: https://www.techrxiv.org/articles/preprint/GPT-4_vs_GPT-3_5_A_Concise_Showdown/22312330

Authorized licensed use limited to: Naresuan University provided by UniNet. Downloaded on September 21,2026 at 03:32:36 UTC from IEEE Xplore. Restrictions apply.

[6] R. McKee, *Story: Substance, Structure, Style and the Principles of Screenwriting*. New York: Harper Collins, 1997.

Authorized licensed use limited to: Naresuan University provided by UniNet. Downloaded on September 21,2026 at 03:32:36 UTC from IEEE Xplore. Restrictions apply.